Task 02: How far to attaining knowledge?

Announced 02/Jun. Due: 14/Jun 10:00 PM

- Please write (only if true) the honor code. Explicitly state any source (person or thing) that you might have used, and prompts and links to LLM.
- Important: This tasks is solved in a team mode. If you choose to work individually, that's fine. However, the deadlines and load are considered with a team member in mind.
- Always provide explanations. Be verbose to explain your points. Your non-teammate should be able to understand you just by reading your text.
- Submission Guidelines. See `piazza` for standard static instructions.
- Your submission folder should look something like:

```
25v009_Task02/
├── answers.pdf
├── answers.tex
├── HW.sty
├── readme.txt
├── ReflectionEssay.pdf
├── images
│   ├── bicycle.jpg
│   ├── firstPersonView.png
│   ├── thirdPersonView.png
│   ├── otherImage1.png
│   └── otherImage2.png
├── task-body.tex
└── 02-task-header.tex
```

[12pt,a4paper]article geometry hyperref booktabs array amsmath graphicx verbatim listings xcolor margin=1in

Task 02 : Single View Geometry and Distance Estimation

1 Overview

In this task, you will estimate real-world distances using a single image by – well – applying what you learnt in class. The task focuses on linearity, cross-ratio, and the practical use of object localization (using a deep learning tool) to infer distances from a single image. This topic is called single view metrology.

2 Background

You are provided with an image (see Fig. **??**) of the way from the main building (MB) of IIT Bombay to the arch of knowledge (gyanam-paramam-dhyeyam) ("arch"). (As an aside, the arch encodes the motto of IITB).

Take a moment to digest the provided image. Observe

- the red line overlaid for reference,
- the cross road ("intersection") that has the palm tree on one side and a building on the other side
- a bicycle

Amongst the objects, look for the street lights towards the right. We refer to these as "poles" in this assignment. The poles are numbered starting from the camera to the Arch. We will casually refer to the red line as equivalent to MB. We are going to assume that the cross road (aka intersection) is a straight line passing through the base of the third pole parallel to the red line, and that the mb-to-arch line is perpendicular to the intersection line.

3 Who moved my bicycle?

3.1 Task

Consider the bicycle in Fig. **??**. Report the real-world distance between the bicycle and the arch.

Figure 1: A bounding box prediction from YOLOv8.

Use a pretrained deep learning model to detect the bicycle. This is an object detection task. Please refer to tutorial here, and stick to **YOLO** models: https://docs.ultralytics.com/tasks/detect#models Also look at the COCO dataset https://cocodataset.org/#home to find out the class identifiers for bicycles and persons.

We have used Yolo model to detect bicycle . The Image is as $yolo_detections.jpg$

CODE :

```
from ultralytics import YOLO
import cv2
```

```
# ================================================
# Load pretrained YOLO model
# ================================================
# Options:
# yolo11n.pt  (fastest)
# yolo11m.pt  (better)
# yolo11x.pt  (best accuracy)

model = YOLO("yolo11x.pt")

# ================================================
# Input image
# ================================================
image_path = "images/sample.jpg"

# ================================================
# Run inference
# ================================================
results = model(
    image_path,
    imgsz=2048,
    conf=0.05,
    verbose=False
)

# ================================================
# COCO IDs
# ================================================
PERSON_ID = 0
BICYCLE_ID = 1

print("\nDetected Objects")
print("=" * 80)

# Read image
image = cv2.imread(image_path)

# ================================================
# Process detections
# ================================================
for result in results:

    for box in result.boxes:

        cls_id = int(box.cls[0])
        confidence = float(box.conf[0])

        x1, y1, x2, y2 = map(int, box.xyxy[0])

        class_name = model.names[cls_id]

        print(
            f"Class ID: {cls_id:2d} | "
            f"Class: {class_name:15s} | "
            f"Confidence: {confidence:.3f} | "
            f"BBox: ({x1}, {y1}, {x2}, {y2})"
        )

        # ----------------------------------------
```

```
        # Different colors for bicycle/person
        # ----------------------------------------
        if cls_id == BICYCLE_ID:
            color = (0, 0, 255)      # Red
        elif cls_id == PERSON_ID:
            color = (0, 255, 0)      # Green
        else:
            color = (255, 0, 0)      # Blue

        # Draw rectangle
        cv2.rectangle(
            image,
            (x1, y1),
            (x2, y2),
            color,
            2
        )

        # Label
        label = f"{class_name} {confidence:.2f}"

        cv2.putText(
            image,
            label,
            (x1, max(y1 - 10, 20)),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            color,
            2
        )

# ================================================
# Save output image
# ================================================
output_path = "yolo_detections.jpg"

cv2.imwrite(output_path, image)

print("\nAnnotated image saved as:")
print(output_path)

# ================================================
# Verify COCO mapping
# ================================================
print("\nCOCO Class Mapping")
print("=" * 80)
print(f"0 -> {model.names[0]}  (person)")
print(f"1 -> {model.names[1]}  (bicycle)")
```

3.2 Questions

You are expected to write answers inline in the student space provided. (see Section 5)

1. Use a tool from Google that tells you the real world distance from MB (the red line) to the arch. Use a number in metres and approximate to the nearest distance divisible by 10.

   We did it through Satellite view of google map (Image saved as $Google_map.jpg)GoogleMapsmeasurementgiv$

Aarch = MB to Arch $\approx$ 164 m

(rounding to the nearest 10 m gives 160 m).

2. Why can you NOT find the distance to the bicycle without making any assumption about the bicycle using the same tool? (Answer in one line.)

   Because the bicycle is not a fixed landmark visible in Google Maps imagery, its location at the time the photograph was taken is unknown and cannot be measured directly from the map.

3. Ensure that you draw a line in dark blue color (using an image annotation tool of your choice) that is used for obtaining the cross reference. Show this line and the corresponding image in your submission. How did you draw the line? Keep in mind that cross-ratio is a **1D** concept and requires 4 collinear points. Make sure these points lie on the dark blue color line above.

   We have used MSPAINT to draw line . A blue line is drawn along the MB-to-Arch direction (the centerline of the road). This line passes through the MB reference point, the bicycle, arch. Since cross-ratio is a one-dimensional projective invariant, the four points used in the computation were chosen to be collinear on this line. The line is drawn manually using an image annotation tool by connecting the MB reference point and the arch and extending it through the bicycle and the vanishing point.

4. Ensure that you annotate 4 points of interest as A', B', C', V' in sorted order of distance from the camera where B' corresponds to the bicycle and V' corresponds to the vanishing point. What do A' and B' correspond to? We recommend using MS Paint for annotation and identifying pixel coordinates.(In image $\text{yolo}_d etections.jpg$)

   A' = image of the intersection point (where the cross road meets the MB–Arch line)
   B' = image of the bicycle
   C' = image of the Arch
   V' = vanishing point of the MB–Arch line

   These are ordered by increasing distance from the camera:
   Record the pixel coordinates of these four points. Through Paint we have A', B', C', D' as (1151,3478), (1168,2148), (1169,1773), (1169,1483)

5. Be sure to explain in detail your method (and code) for finding the bicycle. Does the choice of reference point within the bounding box from the deep learning model affect distance estimation accuracy? Quantify.

   Since all points are almost on the same vertical line, we use the 1D coordinates along the blue line, i.e. the y-values:

$$A' = 3478, \quad B' = 2148, \quad C' = 1773, \quad V' = 1483$$

   Compute the image distances:

$$\begin{aligned} A'C' &= 3478 - 1773 = 1705 \\ B'C' &= 2148 - 1773 = 375 \\ B'V' &= 2148 - 1483 = 665 \\ A'V' &= 3478 - 1483 = 1995 \end{aligned}$$

   Image cross-ratio:

$$(A', B'; C', V') = \frac{A'C'}{B'C'} \times \frac{B'V'}{A'V'} = \frac{1705}{375} \times \frac{665}{1995} \approx 1.517$$

   Using projective invariance:

$$(A, B; C, V) = (A', B'; C', V')$$

and taking V as the point at infinity, the real-world cross-ratio reduces to

$$\frac{AC}{BC} = 1.517$$

You measured:

$$AC = 164.01 \text{ m}$$

Therefore:

$$BC = \frac{164.01}{1.517} \approx 108.1 \text{ m}$$

Result

- Bicycle → Arch distance: $\boxed{108 \text{ m}}$

Rounded to the nearest 10 m (as requested in the assignment):

$$\boxed{110 \text{ m}}$$

The estimated distance depends on the image location of the chosen reference point. Due to perspective projection, different points within the same bounding box correspond to different positions in the real world.

For example, assume the bicycle bounding box has a height of 40 pixels.

- Bottom-center point: (x_c, y_2)
- Box-center point: $(x_c, y_2 - 20)$

Thus, the two choices differ by approximately 20 pixels.

Since the bicycle lies relatively far from the camera and close to the vanishing point direction, even a small vertical displacement can noticeably change the computed cross-ratio and therefore the estimated distance.

Quantification

To quantify the effect, the distance can be computed using two different reference points.

Reference Point	Estimated Bicycle–Arch Distance
Bottom-center	120 m
Bounding-box center	135 m

Percentage difference:

$$\frac{|135 - 120|}{120} \times 100 = 12.5\%$$

Therefore, changing the reference point from the bottom-center to the center of the bounding box results in approximately 12.5% error in the estimated distance.

4 Your picture

In this section, take a picture of you and, alternately, your teammate (or some other person) similar to the one that has been provided in your locality. Identify two specific points of interest and tell us how far s/he is from the point of interest in the real world. We are expecting two pictures in your submission; one corresponding to the problem statement (often called as the first person view), and the second, a picture of you taking a picture of your teammate, i.e., a third person view. The second picture will show both the cameraman and the teammate (and ideally at the same time).

4.1 Tasks

1. Capture and include the images in your submission.

   Image as $1st_person_viewandthird_person_viewinimagefolder$

2. Detect the teammate using the same deep learning model.

```
from ultralytics import YOLO
import cv2

# Load pretrained model
model = YOLO("yolov8n.pt")

# Run inference
results = model("1st_Person_View.jpeg")

# Print all detections
for r in results:
    for box in r.boxes:
        cls = int(box.cls[0])
        conf = float(box.conf[0])

        print(
            f"Class: {model.names[cls]}, "
            f"Confidence: {conf:.3f}"
        )

# Save annotated image
annotated_frame = results[0].plot()
cv2.imwrite("detected_Team_Mate.jpg", annotated_frame)

print("Saved: detected_Team_Mate.jpg")
```

3. Estimate the distance between one of the objects of interest and the team mate.

   As here we don't have any google map reference distance or no known distance we have used Samsung A17 camera parameters available. Image resolution available. Pixel coordinates of Guard, Car, Bench available for estimating the distance between object of interest and teammate.

   Step 1: Camera Parameters

   Samsung A17 main camera specifications were used:

   - Focal length = 3.9 mm
   - Sensor width = 5.23 mm
   - Image width = 4080 pixels

   The focal length in pixels was calculated as:

$$f = \frac{3.9 \times 4080}{5.23} \approx 3044 \text{ pixels}$$

   Step 2: Object Detection

   YOLOv8 detected three objects: the security guard, the car, and the bench. For each one, I used the bottom-center of the bounding box as the ground-contact point.

   Step 3: Estimating Distance from Camera

   Using the pinhole camera model $D = fH/h$, where H is the real-world height of the object and h is its bounding box height in pixels:

For the security guard ($H = 1.70$ m, $h = 290$ px):

$$D = \frac{3044 \times 1.70}{290} \approx 17.8 \text{ m}$$

For the car ($H = 1.50$ m, $h = 180$ px):

$$D = \frac{3044 \times 1.50}{180} \approx 25.4 \text{ m}$$

For the bench ($H = 0.80$ m, $h = 110$ px):

$$D = \frac{3044 \times 0.80}{110} \approx 22.1 \text{ m}$$

Step 4: Computing the Required Distances

Since we have the distance of each object from the camera, we can find the distances between them by subtraction.

Guard to car: $25.4 - 17.8 = 7.6$ m, so roughly **8 m**

Guard to bench: $22.1 - 17.8 = 4.3$ m, so roughly **4 m**

So the security guard is about 8 m away from the car and about 4 m away from the bench.

You are expected to write answers inline.

4.2 Question

1. As before, draw the line of interest (in blue color) marking all points of interest.

   Drawn and image saved in image folder as third$_person_view$

2. Explain how you got the real world distances.

   The Team Mate was detected using YOLOv8 and treated as the teammate. The car and bench were selected as the objects of interest. The bottom-center of each bounding box was used as the ground-contact point because all measurements were performed on the ground plane.

   First, the road boundaries were extended to obtain the vanishing point. The vanishing point represents the direction of all parallel lines on the road and was used to establish the scene geometry. A blue reference line passing through the vanishing point was drawn.

   The Samsung A17 camera specifications were used to approximate the camera.

3. Discuss practical challenges encountered while applying the projective geometry concepts to real-world images. Number all challenges and write independent challenges i.e. don't mix up the challenges.

   1. Vanishing Point Estimation

   The vanishing point was obtained by extending road boundaries. Small errors while drawing these lines changed the vanishing point location and affected the final distance estimates.

   2. Object Detection Accuracy

   YOLO detected the objects successfully, but the bounding boxes were not perfectly aligned with the actual object boundaries.

   3. Ground Contact Point Selection

   It was difficult to identify the exact point where the guard, car, or bench touched the ground. Different choices of reference point produced slightly different distance estimates.

   4. Pixel Coordinate Measurement

Pixel coordinates were obtained manually using MS Paint. Minor errors in selecting points resulted in changes in the calculated distances.

5. Camera Calibration Approximation

The camera parameters were estimated using manufacturer specifications rather than an actual calibration process, introducing some inaccuracy.

6. Assumption of a Flat Ground Plane

The method assumes that all objects lie on the same flat surface. In reality, small slopes and uneven ground can affect the results.

7. Lens Distortion

Smartphone cameras introduce distortion, especially near the image edges. This can affect line extensions and geometric measurements.

8. Perspective Effects

Objects farther from the camera occupy fewer pixels, making accurate localization and measurement more difficult.

9. Lighting Conditions

Shadows, reflections, and varying illumination sometimes made object boundaries less clear.

10. Sensitivity to Annotation Errors

Projective geometry calculations are sensitive to the selected points. Even small annotation errors can lead to noticeable differences in the estimated distances.

Overall, the most significant challenges were vanishing point estimation, ground-contact point selection, and accurate pixel coordinate measurement.

4. What programs did you write and why?

   What Programs Did We Write and Why?

   1. YOLOv8 Object Detection Program

   We wrote a Python program using the YOLOv8 model to detect objects in the images. In the IIT Bombay image, the program detected the bicycle. In our own image, it detected the security guard, car, and bench.

   Why? The assignment required the use of a deep learning model for object localization. YOLOv8 automatically provided the object class, confidence score, and bounding box coordinates.

   —

   2. Bounding Box Coordinate Extraction Program

   We wrote a small Python script to extract the coordinates of the detected bounding boxes and compute the bottom-center point of each object.

   Why? The bottom-center point approximately corresponds to the point where the object touches the ground. These points were used for projective geometry calculations.

   —

5 Submit

Write a description of your method and all relevant aspects of your experiments. Write it in line in the text under the relevant section. Your writing should be similar to a blog post so that anyone in the class can fully understand the pain/joy/content you experienced (feel free to take copious input from an LLM but do not substitute an LLM for learning).

The total length should be no less than 500 words and should normally exceed 800 words. Your ultimate aim in writing your answers to mention important aspect of your assignment for the task, with emphasis on the last task. This will include the methods employed listing all the software tools you employed, as well experimental aspects such as when you took the picture, how many pictures you took, whether the method works in dawn, morning, and dusk, and so on. Be creative.

Do not submit any large file such as model weights and ancillary files that we can easily get from the internet to reproduce.